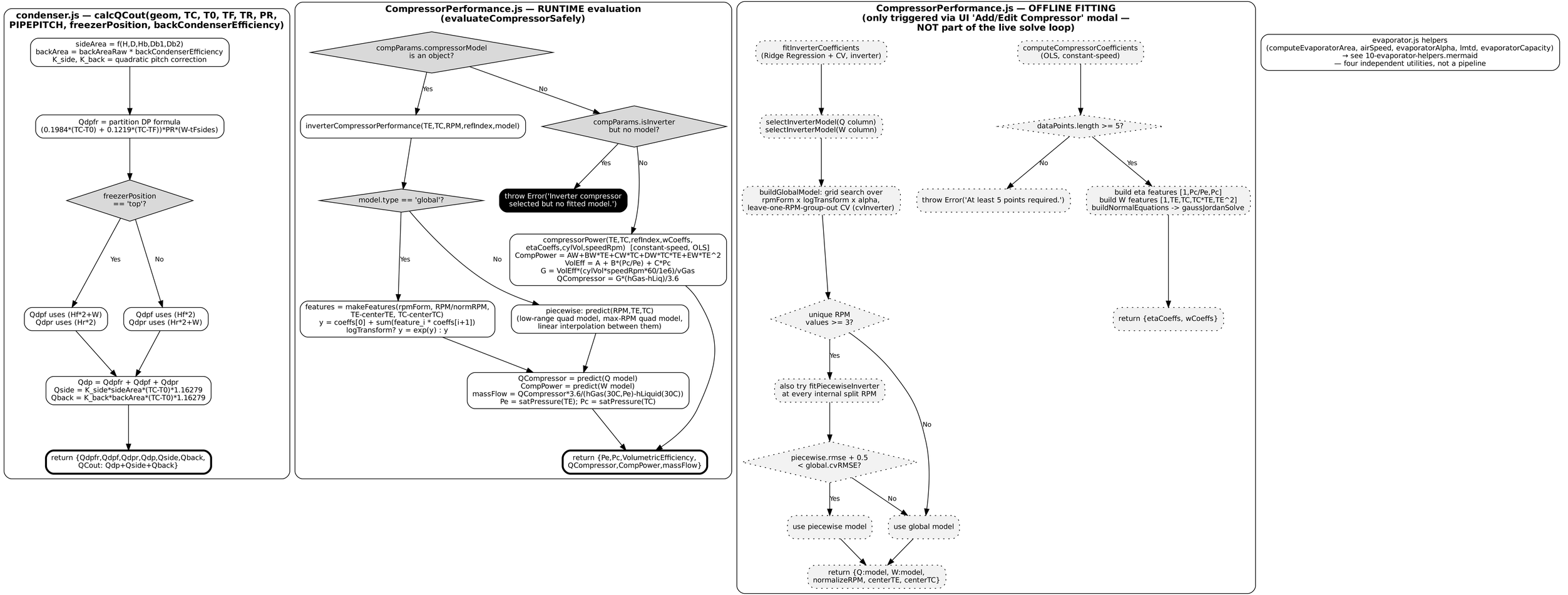
condenser.js — calcQCout(geom, TC, T0, TF, TR, PR, PIPEPITCH, freezerPosition, backCondenserEfficiency)
sideArea = f(H,D,Hb,Db1,Db2)
backArea = backAreaRaw * backCondenserEfficiency
K_side, K_back = quadratic pitch correction
Qdpfr = partition DP formula
(0.1984*(TC-T0) + 0.1219*(TC-TF))*PR*(W-tFsides)
freezerPosition
== 'top'?
Yes
No
Qdpf uses (Hf*2+W)
Qdpr uses (Hr*2)
Qdpf uses (Hf*2)
Qdpr uses (Hr*2+W)
Qdp = Qdpfr + Qdpf + Qdpr
Qside = K_side*sideArea*(TC-T0)*1.16279
Qback = K_back*backArea*(TC-T0)*1.16279
return {Qdpfr,Qdpf,Qdpr,Qdp,Qside,Qback,
QCout: Qdp+Qside+Qback}
CompressorPerformance.js — RUNTIME evaluation
(evaluateCompressorSafely)
compParams.compressorModel
is an object?
Yes
No
inverterCompressorPerformance(TE,TC,RPM,refIndex,model)
compParams.isInverter
but no model?
Yes
No
model.type == 'global'?
throw Error('Inverter compressor
selected but no fitted model.')
Yes
No
compressorPower(TE,TC,refIndex,wCoeffs,
etaCoeffs,cylVol,speedRpm) [constant-speed, OLS]
CompPower = AW+BW*TE+CW*TC+DW*TC*TE+EW*TE^2
VolEff = A + B*(Pc/Pe) + C*Pc
G = VolEff*(cylVol*speedRpm*60/1e6)/vGas
QCompressor = G*(hGas-hLiq)/3.6
features = makeFeatures(rpmForm, RPM/normRPM,
TE-centerTE, TC-centerTC)
y = coeffs[0] + sum(feature_i * coeffs[i+1])
logTransform? y = exp(y) : y
piecewise: predict(RPM,TE,TC)
(low-range quad model, max-RPM quad model,
linear interpolation between them)
QCompressor = predict(Q model)
CompPower = predict(W model)
massFlow = QCompressor*3.6/(hGas(30C,Pe)-hLiquid(30C))
Pe = satPressure(TE); Pc = satPressure(TC)
return {Pe,Pc,VolumetricEfficiency,
QCompressor,CompPower,massFlow}
CompressorPerformance.js — OFFLINE FITTING
(only triggered via UI 'Add/Edit Compressor' modal —
NOT part of the live solve loop)
fitInverterCoefficients
(Ridge Regression + CV, inverter)
selectInverterModel(Q column)
selectInverterModel(W column)
buildGlobalModel: grid search over
rpmForm x logTransform x alpha,
leave-one-RPM-group-out CV (cvInverter)
unique RPM
values >= 3?
Yes
No
also try fitPiecewiseInverter
at every internal split RPM
piecewise.rmse + 0.5
< global.cvRMSE?
Yes
No
use piecewise model
use global model
return {Q:model, W:model,
normalizeRPM, centerTE, centerTC}
computeCompressorCoefficients
(OLS, constant-speed)
dataPoints.length >= 5?
No
Yes
throw Error('At least 5 points required.')
build eta features [1,Pc/Pe,Pc]
build W features [1,TE,TC,TC*TE,TE^2]
buildNormalEquations -> gaussJordanSolve
return {etaCoeffs, wCoeffs}
evaporator.js helpers
(computeEvaporatorArea, airSpeed, evaporatorAlpha, lmtd, evaporatorCapacity)
→ see 10-evaporator-helpers.mermaid
— four independent utilities, not a pipeline